

D S T Q Q S S
D L M M J V S

Disciplina: Aprendizado Profundo.

Período: 2025.2

Professor: Eliezer de Souza de Silva.

Aluno: Davi Teixeira Silva.

Lista de Exercícios 2

Parte 1: Exercícios Teóricos e Práticos.

Exercício 1: Teoria da Convergência

Responda brevemente:

1. Adam é frequentemente o otimizador padrão, mas em alguns trabalhos de pesquisa, o bom e velho SGD com Momento alcança um erro de generalização final ligeiramente melhor. Com base na nossa discussão sobre mínimos planos vs. afiados, formule uma hipótese para explicar por que isso acontece. (Dica: pense em como a taxa de aprendizado adaptativa do Adam pode se comportar em diferentes geometrias de mínimos).

Hipótese: Mínimos Planos vs. Afiados

1. O objetivo da generalização:

Modelos com boa generalização (bem desajustados em dados não vistos) tendem a convergir para mínimos planos. Mínimos planos são regiões onde a função de perda permanece baixa mesmo com pequenas variações nos parâmetros do modelo. Isso indica que o modelo é mais robusto a flutuações nos dados e menos propenso a overfitting. Mínimos afiados, por outro lado, são pontos de perda muito baixa, mas que aumentam rapidamente com pequenas mudanças nos parâmetros.

Jandaia

D S T Q Q S S
D L M M J V S

2. O comportamento do Adam (taxa de aprendizado adaptativas):

- O Adam ajusta a taxa de aprendizado de cada parâmetro com base nas médias móveis dos gradientes passados (primeiro e segundo momentos).
- Em regiões de mínimos planos, os gradientes tendem a ser pequenos. O Adam pode aumentar a taxa de aprendizado para tentar sair dessa região ou pode manter os parâmetros "presos" em um ponto específico, pois a taxa de aprendizado adaptativa normaliza os gradientes, tornando mais difícil "sentir" a largura do mínimo.
- Em regiões de mínimos afiados, os gradientes podem ser grandes e variáveis. O Adam pode se adaptar bem a essa geometria, encontrando o ponto exato do mínimo afiado com eficiência.

3. O comportamento do SGD com Momento (taxa de aprendizado global e fixa):

- Essa taxa de aprendizado fixa e momentum (que ajuda a suavizar as oscilações e a sair de mínimos locais) atuam com um tipo de "arrasto" ou "força inercial" que impede o otimizador de se fixar muito rapidamente em mínimos afiados.
- A inércia do SGD com Momento tende a empur-

D S T Q Q S S
D L M M J V S

nao o otimizador para fora de mínimos muito estreitos e afiados, direcionando-o para as regiões mais amplas e planas da superfície de perda, que como mencionado, estão associadas a uma melhor generalização.

Em resumo:

A taxa de aprendizado adaptativa do Adam permite que ele encontre mínimos (planos ou afiados) mais rapidamente. No entanto, a natureza estática e a taxa de aprendizado global do SGD com momento, especialmente com um bom ajuste dos hiperparâmetros (taxa de aprendizado e momento), atuam como um mecanismo que evita mínimos afiados, favorecendo a convergência para mínimos planos, o que resulta em um erro de generalização final ligeiramente melhor.

2. A partir da inequação de convergência para funções L -suaves e μ -fortemente convexas, $\epsilon_{t+1} \leq (1 - \mu/L)\epsilon_t$, mostre que o número de iterações T necessário para atingir uma precisão ϵ (i.e., $\epsilon_T \leq \epsilon$) é da ordem de $O(\log(1/\epsilon))$.

Passo 1: Definir a Inequação de Convergência Recursiva

A inequação de convergência dada para o erro ϵ_t na iteração t é:

$$\epsilon_{t+1} \leq \left(1 - \frac{\mu}{L}\right) \epsilon_t$$

Jandaia

D S T Q Q S S
D L M M J V S

Podemos definir uma constante de convergência $C = 1 - \mu/L$. Como $\mu > 0$ e L é finito e positivo (propriedades de f de convergência e suavidade), temos $0 < C < 1$. A inequação fica:

$$e_{f+1} \leq C e_f$$

Passo 2: Generalizar a Inequação para T iterações
Aplicando a relação recursivamente a partir do erro inicial e_0 :

$$e_1 \leq C e_0$$

$$e_2 \leq C e_1 \leq C(C e_0) = C^2 e_0$$

...

$$e_T \leq C^T e_0$$

Passo 3: Determinar o Número de Iterações T

O objetivo é encontrar o número de iterações T necessário para que o erro final e_T seja menor ou igual a uma precisão desejada ϵ .

$$e_T \leq \epsilon$$

Substituindo a relação encontrada no Passo 2:

$$C^T e_0 \leq \epsilon$$

$$C^T \leq \frac{\epsilon}{e_0}$$

Passo 4: Usar Logaritmos para Resolver para T

Multiplicamos o logaritmo natural ($\ln$) em ambos os lados da inequação. Como $0 < C < 1$, $\ln(C)$ é um valor negativo. Dividir por um número negativo inverte o sinal da inequação:

$$T \ln(C) \leq \ln\left(\frac{\epsilon}{\epsilon_0}\right)$$

$$T \geq \frac{\ln(\epsilon/\epsilon_0)}{\ln(C)}$$

Podemos reescrever a expressão para melhor clareza, lembrando que $-\ln(C)$ é positivo:

$$T \geq \frac{\ln(\epsilon_0/\epsilon)}{-\ln(C)} = \frac{\ln(\epsilon_0) - \ln(\epsilon)}{-\ln(C)}$$

Passo 5: Determinar a Ordem de Complexidade (Notação O)

A expressão mostra que T é proporcional a $\ln(\epsilon_0/\epsilon)$.

$$T \geq K \cdot \ln\left(\frac{\epsilon_0}{\epsilon}\right), \text{ onde } K = \frac{1}{-\ln(C)} \text{ é uma constante}$$

positiva que depende apenas das propriedades das funções (μ e ϵ). Expandindo o logaritmo:

$$T \geq K \cdot (\ln(\epsilon_0) + \ln(1/\epsilon))$$

$\ln(\epsilon_0)$ é uma constante para um dado problema. O termo dominante que depende da precisão alvo ϵ é $\ln(1/\epsilon)$. Portanto, o número de iterações T cresce logaritmicamente com o inverso da precisão desejada, $1/\epsilon$. Dizemos que a complexidade é de $O(\log(1/\epsilon))$.

D S T Q Q S S
D L M M J V S

Exercício 2: Versão de Gradiente Ruído (Noisy GD)

Neste exercício, vamos explorar uma versão da Versão de Gradiente onde injetamos ruído Gaussiano explicitamente a cada passo de duas formas. A regra de atualização é:

$$\theta_{t+1} \leftarrow \theta_t - \eta (\nabla J(\theta_t) + \epsilon_t), \text{ onde } \epsilon_t \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$$

Parte A: Implementação (Numpy). Crie uma nova classe de otimizador, "Noisy GD", no seu framework Numpy. O construtor deve receber o modelo, a taxa de aprendizagem "learning-rate" e um novo tipo de parâmetro "noise-level" (σ). O método "step" deve implementar a regra de atualização acima.

[Código implementado no jupyter notebook]

Parte B: Análise Teórica

- **B.1.** Mostre que, em expectativa, a atualização do Noisy GD é idêntica à da Versão de Gradiente padrão. Escreva a expressão para a atualização esperada, $E[\theta_{t+1} | \theta_t]$ (use a definição do ruído Gaussiano $E[\epsilon_t] = 0$ e linearidade do valor esperado).

A regra de atualização do Noisy GD é definida como:

$$\theta_{t+1} \leftarrow \theta_t - \eta (\nabla J(\theta_t) + \epsilon_t)$$

Nosso objetivo é calcular o valor esperado (expectativa) do novo parâmetro θ_{t+1} , condicional ao valor atual do parâmetro θ_t .

Jandaia

D S T Q Q S S
D L M M J V S

$$E[\theta_{t+1} | \theta_t] = E[\theta_t - \eta(\nabla J(\theta_t) + \epsilon_t) | \theta_t]$$

Usando a linearidade do valor esperado ($E[aX + bY] = aE[X] + bE[Y]$):

$$E[\theta_{t+1} | \theta_t] = E[\theta_t | \theta_t] - E[\eta \nabla J(\theta_t) | \theta_t] - E[\eta \epsilon_t | \theta_t]$$

Sabemos que:

- θ_t , η (taxa de aprendizagem) e $\nabla J(\theta_t)$ (o gradiente no ponto atual) são constantes ou valores fixos condicionados a θ_t . A expectativa de uma constante é a própria constante.
 - ϵ_t é o ruído Gaussiano, com valor esperado $E[\epsilon_t] = 0$.
- Aplicando essas propriedades:

$$E[\theta_{t+1} | \theta_t] = \theta_t - \eta \nabla J(\theta_t) - \eta E[\epsilon_t]$$

$$E[\theta_{t+1} | \theta_t] = \theta_t - \eta \nabla J(\theta_t) - \eta \cdot 0$$

A expressão final para a atualização esperada é:

$$E[\theta_{t+1} | \theta_t] = \theta_t - \eta \nabla J(\theta_t)$$

Esta é exatamente a regra de atualização da Descida de Gradiente (GD) padrão.

- B.3. O que essas propriedades nos dizem sobre o comportamento do otimizador?

1. Convergência Média:

Embora cada passo individual do Noisy GD seja ruidoso e aleatório, "em média" ou "em expectativa", o otimizador está seguindo exatamente o mesmo caminho que o otimizador GD padrão (determinístico).

Jandaia

Isso significa que, se você rodar o treinamento com Noise-GD muitas vezes, a média dos caminhos de otimização resultaria no caminho do GD padrão. A injeção de ruído não introduz um viés (bias) no processo de otimização; ela apenas aumenta a variância de cada passo.

2. Mecanismo de Escape de Mínimos Locais (e favorecimento de Mínimos Planos):

Apesar de expectativa ser a mesma do GD padrão, a variabilidade introduzida pelo ruído Gaussiano tem um efeito prático crucial:

- O ruído atua como um "empurrão" aleatório em cada iteração.
- Isso ajuda a otimizador a escapar de mínimos locais rasos ou de regiões de saddle points (pontos de sela), onde um GD padrão pode ficar preso.
- O ruído incentiva o explorador a "saltar" pela superfície de perda, o que, na prática, muitas vezes leva a regiões de mínimos planos.
- A exploração de mínimos planos está diretamente ligada a uma generalização do modelo para dados não vistos.

Em resumo: o Noise-GD é, em média, igual ao GD padrão, mas o ruído o torna um otimizador estocástico mais robusto na prática, favorecendo a exploração da superfície de perda e a busca por soluções com boa generalização.

D S T Q S S
D L M M J V S

Parte C: Análise Prática

Use a função de custo 2D não-convexa do laboratório para comparar visualmente os caminhos de minimização de:

1. GD Padrão (que pode ser simulado com "Noisy GD" e "noise-level=0").
2. Noisy GD com um diferentes níveis de ruído moderado (e.g., "noise-level" = [0.01, 0.05, 0.1, 0.5, 1.0, 5.0]).

Plote os caminhos sobre a mesma paisagem de custo. Diferente as diferenças. O ruído ajudou a otimizador a explorar a paisagem ou a escapar de mínimos locais?

1. GD Padrão (noise-level=0.0)

- O caminho do GD padrão (linha azul) é suave e determinístico.
- Ele segue a direção exata do gradiente descendente a cada passo.
- Dependendo do ponto inicial e da superfície (a função de Beale tem vales longos e estreitos), ele pode ficar "preso" em um mínimo local ou demorar muito para chegar aos vales até o mínimo global.

2. Noisy GD (Níveis de Ruído Moderado)

- Os caminhos do Noisy GD são mais erráticos e ríspidos que antes.
- Ruído Baixo (0.01, 0.05): O caminho geral ainda segue a direção do gradiente, mas o ruído adiciona pequenas perturbações que ajudam a "explorar" brevemente a área circundante, potencialmente evitando ficar preso em mínimos locais muito rasos.

D S T Q Q S S
D L M M J V S

- Ruído Moderado (0.1, 0.5): O comportamento de exploração é mais pronunciado. O otimizador faz "saltos" maiores. Isso é especialmente útil para sair de regiões de saddle points ou mínimos locais.

3. Ruído Alto (e.g., 1.0 e 5.0)

- Se o nível de ruído for muito alto em comparação com a magnitude do gradiente, o otimizador pode começar a se mover quase aleatoriamente.
- Isso impede que ele converja eficientemente para o mínimo global; ele apenas "vagueia" pela superfície de custo, raramente encontrando a solução ótima, mesmo que explore bem o espaço.

Conclusão:

Sim, o ruído ajudou o otimizador a explorar a paisagem. O principal benefício do ruído é a exploração robusta. Ele impede que o otimizador se fixe prematuramente em soluções subótimas (mínimos locais aprados). O nível de ruído ideal age como um balanço entre a exploração da superfície de perda e a exploração (convergência para o mínimo global). Na prática, esse efeito estocástico é crucial para encontrar soluções que generalizem bem.

Parte 2: Implementação e Visualização.

[Código implementado no jupyter notebook]